

Name:	ID:	Section:
-------	-----	----------

Question 1 [8 Marks]

In an emergency hospital, patients are treated according to their priority. The hospital uses a **Max Heap implemented with an array** to manage the patients. Each patient is represented as: (ID, Priority, WaitingTime)

A patient A has higher priority than patient B according to the following rules:

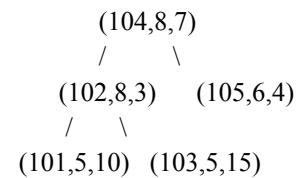
- A patient with a higher Priority is treated first.
- If two patients have the same Priority, the patient with the higher WaitingTime is treated first.
- If both Priority and WaitingTime are the same, the patient with the smaller ID is treated first.

The following **existing Max Heap** is given:

•

index	0	1	2	3	4
Heap	104	102	105	101	103

The heap is represented as:



Write a Java program to remove patients from the heap one by one and print their treatment order. You are given the existing heap, so **you do not need to build the heap**. Your task is to modify the standard **Max Heap heapify() method** so that it maintains the heap according to the hospital's priority rules after each removal.

Use the following array representation for the heap: left child = $2 * i + 1$ right child = $2 * i + 2$

Choose the correct answer from the given options [4 Marks]

1. Consider the following Max Heap: [90, 70, 80, 40, 60, 75, 50] .
The root value is replaced with 35: [35, 70, 80, 40, 60, 75, 50] .
After performing standard Max Heapify at index 0, what will be the resulting array?

- A. [80, 70, 75, 40, 60, 35, 50]
- B. [80, 70, 35, 40, 60, 75, 50]
- C. [75, 70, 80, 40, 60, 35, 50]
- D. [80, 75, 70, 40, 60, 35, 50]

2. Consider the following heapify() method for a Max Heap:

What is the main problem with this implementation?

- A. The left-child formula is incorrect.
- B. The right-child formula is incorrect.
- C. The affected subtree is not heapified after the swap.
- D. largest should always be initialized to 0.

3. Which of the following statements about building a Max Heap using the bottom-up(Swim) heap construction method is correct?

- A. The time complexity is always $O(n \log n)$.
- B. The time complexity is $O(n)$.
- C. The time complexity is $O(\log n)$.
- D. The time complexity is $O(n^2)$.

4. Consider the following valid Max Heap [100, 90, 80, 70, 60, 50, 40, 30, 20] Suppose the element 70 is deleted and replaced by the last element 20: [100, 90, 80, 20, 60, 50, 40, 30]

Which approach is correct for restoring the heap property?

- A. Always move the replacement element downward.
- B. Always move the replacement element upward.
- C. Compare the replacement element with its parent. if necessary move upward, otherwise move downward.
- D. No heapify operation is required.

Determine whether each statement is True or False [3 Marks]

- 1. In a valid Max Heap, if the root element is removed and replaced by the last element, the heap property can only be violated along the downward path from the root.
- 2. In a Max Heap containing distinct elements, the second-largest element can appear at any internal node as long as it is greater than its children.
- 3. For an array of n elements, performing heapify() separately on every internal node from top to bottom always guarantees an $O(n)$ heap construction.

(only for question 2)

```
void heapify(int[] arr, int n, int i) {  
    int largest = i;  
    int left = 2 * i + 1;  
    int right = 2 * i + 2;  
  
    if (left < n && arr[left] > arr[largest])  
        largest = left;  
  
    if (right < n && arr[right] > arr[largest])  
        largest = right;  
  
    if (largest != i) {  
        int temp = arr[i];  
        arr[i] = arr[largest];  
        arr[largest] = temp;  
    }  
}
```